

Raport końcowy projektu MyTutor

- **Nazwa projektu:** MyTutor - Szkoła Korepetycji
- **Temat projektu:** Projekt i implementacja aplikacji webowej wspierającej zarządzanie szkołą korepetycji.
- **Zespół projektowy:**
 - Marcel Gładysz
 - Adam Nowak
 - Krzysztof Wiewióra
- **Stos technologiczny:** Java, Spring Boot (Backend) / React, Vite, TypeScript (Frontend)
- **Cel projektu:** Stworzenie scentralizowanego, wydajnego i bezpiecznego systemu ułatwiającego administrację szkołą korepetycji, obsługę harmonogramów oraz komunikację między administratorami, korepetytorami a kursantami.

1. Wprowadzenie

Aplikacja MyTutor stanowi odpowiedź na **problem** rosnącego chaosu organizacyjnego, z którym borykają się rozrastające się placówki edukacyjne oraz prywatne szkoły korepetycji. Tradycyjne metody zarządzania harmonogramem (np. arkusze kalkulacyjne) generują ryzyko błędów ludzkich, takich jak nakładanie się zajęć czy problemy z komunikacją o odwołanych lekcjach.

Grupa docelowa systemu to przede wszystkim:

1. Właściciele i administratorzy szkół korepetycji.
2. Nauczyciele/korepetytorzy prowadzący zajęcia.
3. Kursanci (uczniowie) korzystający z usług placówki.

Główną motywacją stworzenia aplikacji była chęć cyfryzacji i automatyzacji procesów administracyjnych, co pozwoli zaoszczędzić czas kadry zarządzającej oraz poprawić jakość i przejrzystość usług dla klientów.

2. Zakres projektu

Funkcje zrealizowane w ramach projektu:

- Autoryzacja i zarządzanie profilami użytkowników (Admin, Korepetytor, Kursant).
- Zarządzanie grupami szkoleniowymi i przypisywanie do nich kursantów.
- Tworzenie oraz modyfikacja harmonogramu zajęć.
- Algorytm wykrywania konfliktów terminów (zapobieganie podwójnemu przypisaniu nauczyciela lub nakładaniu się lekcji kursanta).
- System powiadomień wewnętrznych (in-app).
- Kontrola dostępu do zasobów według ról (Role-Based Access Control).

Funkcje świadomie wyłączone z zakresu (Out-of-scope):

- Integracja z zewnętrznymi bramkami płatności.
- Moduł do prowadzenia zajęć online (wideokonferencje).
- Zewnętrzne systemy powiadomień (np. e-mail, SMS, push) - zaimplementowano wyłącznie system wewnętrzny.

3. Przebieg prac projektowych (Podsumowanie Sprintów)

Proces wytwórczy aplikacji MyTutor oparty był o metodykę zwinną, co pozwoliło na iteracyjne dostarczanie wartości i systematyczną weryfikację założeń z użytkownikami.

Wczesne fazy i projektowanie

W ramach początkowych sprintów opracowano szczegółowe wymagania funkcjonalne i нефункционалне systemu w postaci kompletnych kryteriów akceptacji oraz scenariuszy testowych, które zostały zapisane w formacie GWT. Dodatkowo przygotowano flow użytkownika w postaci diagramów aktywności. Przedstawiały one przebieg najważniejszych procesów realizowanych w systemie przez administratorów, korepetytorów oraz kursantów. Na podstawie tych diagramów wykonano interaktywny prototyp interfejsu użytkownika w wersji low-fidelity, co umożliwiło wizualizację struktury aplikacji, nawigacji oraz podstawowych funkcjonalności bez szczegółowej warstwy graficznej.

Sprint 3 - Weryfikacja UI i Dokumentacja Techniczna

Celem trzeciego sprintu było zweryfikowanie poprawności zaprojektowanego interfejsu użytkownika oraz przygotowanie kompletnej dokumentacji technicznej stanowiącej podstawę implementacji systemu MyTutor. Zrealizowane prace podzielono pomiędzy członków zespołu:

- **Badanie interfejsu użytkownika (Marcel Gładysz):** Przeprowadzono badanie użyteczności prototypu z udziałem kursantów, korepetytorów oraz administratorów. Badanie wykazało, że podstawowe zadania mogą być realizowane w sposób intuicyjny, a nawigacja jest czytelna i spójna. Jednocześnie zidentyfikowano potrzebę rozbudowy formularzy oraz dodania bardziej widocznych komunikatów potwierdzających wykonanie operacji.
- **Finalizacja prototypu UI (Krzysztof Wiewióra):** Na podstawie wyników badań dopracowano interaktywny prototyp aplikacji. Ujednolicono sposób nawigacji pomiędzy ekranami, uporządkowano układ widoków oraz zweryfikowano kompletność funkcjonalności dostępnych dla ról w systemie.
- **Architektura i diagramy (Adam Nowak):** Przygotowano architekturę logiczną aplikacji z podziałem na frontend, backend i bazę danych oraz opracowano diagram pakietów. Opracowano diagram klas opisujący model domenowy, relacje pomiędzy obiektami, logikę biznesową i dostęp do danych. Dodatkowo przygotowano diagramy sekwencji dla najważniejszych procesów (zarządzanie użytkownikami, grupami, harmonogramem zajęć, zapisami na zajęcia oraz powiadomieniami) przedstawiające sposób przetwarzania żądań pomiędzy interfejsem, warstwą usług, repozytoriami i bazą danych.

4. Architektura systemu

System MyTutor oparty jest na klasycznej **architekturze trójwarstwowej** (Three-Tier Architecture), rozdzielającej interfejs użytkownika, logikę biznesową i warstwę dostępu do danych:

- **Frontend (Warstwa Prezentacji):** Zbudowany w oparciu o bibliotekę **React** z wykorzystaniem **TypeScript** dla silnego typowania i minimalizacji błędów czasu wykonywania. Jako narzędzie budujące wykorzystano **Vite**, co znacząco przyspieszyło proces developmentu.
- **Backend (Warstwa Logiki Biznesowej):** Zaimplementowany w języku **Java** przy

użyciu frameworka **Spring Boot**. Odpowiada za autoryzację, walidację danych, algorytmy wykrywania konfliktów oraz przetwarzanie zapytań z frontendu.

- **Baza Danych (Warstwa Danych):** W systemie zastosowano relacyjną bazę danych do przechowywania powiązań między użytkownikami, grupami a harmonogramem.
- **Komunikacja:** Warstwy komunikują się ze sobą poprzez architekturę **REST API**, wymieniając dane w formacie JSON.

Uzasadnienie wyboru technologii: Wybrano technologie o ugruntowanej pozycji rynkowej. Spring Boot gwarantuje stabilność, bezpieczeństwo i łatwszą skalowalność na backendzie, podczas gdy React w połączeniu z TypeScriptem zapewnia dynamiczny i bezbłędny interfejs użytkownika.

5. Model danych

Opierając się na specyfikacji, model danych (odzwierciedlony w diagramach klas backendu) opiera się na następujących głównych encjach:

- **User (Użytkownik):** Przechowuje dane logowania, imię, nazwisko oraz rolę (Admin, Tutor, Student).
- **Group (Grupa):** Reprezentuje klasę/grupę przedmiotową. Posiada limit miejsc oraz powiązanego korepetytora.
- **Lesson (Lekcja):** Konkretnie wydarzenie w czasie. Posiada datę rozpoczęcia, czas trwania, status i przypisaną grupę.
- **Enrollment (Zapis):** Encja łącząca (tabela asocjacyjna) w relacji Many-to-Many między użytkownikiem (Kursantem) a Grupą.
- **Notification (Powiadomienie):** Zawiera treść komunikatu, datę wygenerowania, nadawcę oraz odbiorcę (lub listę odbiorców).

Kluczowe relacje: Jedna grupa może mieć wiele zaplanowanych lekcji (1:N). Jeden korepetytor może uczyć wiele grup (1:N). Kursanci przypisani są do grup wielokrotnie (M:N).

6. Logika biznesowa

System przetwarza kluczowe zasady działania szkoły na kod:

- **Zarządzanie grupami i przypisywanie korepetytorów:** Tylko administrator może tworzyć nowe grupy i delegować do nich odpowiednich nauczycieli na podstawie ich specjalizacji.
- **Zapisy i rezygnacje:** Kursant może zapisać się do grupy, o ile nie przekroczono w niej maksymalnego limitu miejsc. Rezygnacja jest możliwa do określonego czasu przed startem zajęć.
- **Wykrywanie konfliktów terminów (Kluczowa funkcja):** Algorytm na backendzie przed utworzeniem lekcji sprawdza w bazie, czy dany korepetytor lub sala (jeśli jest uwzględniana) nie mają już w tym samym oknie czasowym innych zaplanowanych zajęć. W przypadku kolizji system rzuca błąd, a na frontendzie wyświetlany jest stosowny komunikat.
- **Powiadomienia:** Tworzenie i odczytywanie asynchronicznych wiadomości kierowanych wewnątrz systemu do konkretnych ról.

7. Interfejs użytkownika

Zgodnie z projektem makiety interfejs kładzie nacisk na ergonomię i nowoczesny design.

- **Główne widoki:**
 - **Dashboard główny:** Ekran powitalny podsumowujący dzisiejsze zajęcia i

- nieprzeczytane powiadomienia.
- **Widok Kalendarza:** Interaktywny plan zajęć, wyświetlający lekcje w podziale na dni i tygodnie. Różne kolory oznaczają status zajęć (np. zaplanowane, odwołane).
- **Panel Zarządzania (tylko Admin):** Tabelaryczny widok kursantów, korepetytorów i grup, pozwalający na łatwe dodawanie/usuwanie rekordów.
- **Przepływy (User Flows):** Administrator loguje się -> przechodzi do zakładki "Grupy" -> klika "Dodaj Grupę" -> wypełnia formularz i przypisuje korepetytora -> Kursant loguje się -> w zakładce "Zapisy" widzi nową grupę -> potwierdza chęć udziału.

8. Testowanie

- **Obszary testowane:** Logika backendu objęta jest zestawem testów jednostkowych (np. za pomocą JUnit i Mockito).
- **Przykładowe scenariusze testowe:**
 1. **Test konfliktów:** Dodanie lekcji X w godzinach 15:00-16:00 dla Korepetytora A. Następnie próba dodania lekcji Y dla tego samego korepetytora w godzinach 15:30-16:30. **Oczekiwany rezultat:** Odrzucenie transakcji, zwrócenie błędu walidacji terminów.
 2. **Test limitów grup:** Próba dodania 11. kursanta do grupy o sztywnym limicie 10 osób. **Oczekiwany rezultat:** Wyjątek przekroczenia rozmiaru grupy.
 3. **Test uprawnień (RBAC):** Próba wywołania chronionego endpointu z użyciem tokenu logowania Kursanta. **Oczekiwany rezultat:** Status HTTP 403 (Forbidden).

9. Zgodność implementacji ze specyfikacją

Po porównaniu założeń ze specyfikacją techniczną z obecnym stanem implementacji:

- **Zrealizowano pomyślnie:** Architektura trójwarstwowa, pełny CRUD operacyjny, kalendarz, system kontroli dostępu.
- **Zgodność w zakresie ograniczeń:** W specyfikacji wyraźnie wskazano uproszczenia - w szczególności powiadomienia opierają się wyłącznie na systemie wewnątrz aplikacji (in-app notifications). Zewnętrzne powiadomienia e-mail, SMS oraz push nie zostały zaimplementowane, co jest całkowicie zgodne z przyjętymi na starcie ograniczeniami MVP.

10. Problemy napotkane podczas realizacji

1. **Model autoryzacji:** Z uwagi na ograniczenia czasowe projektu, zdecydowano się na uproszczony model autoryzacji, bez integracji z zewnętrznymi dostawcami tożsamości.
2. **Złożoność algorytmu konfliktów:** Trudnością techniczną okazało się wydajne sprawdzanie nakładających się przedziałów czasowych. Zamiast obciążać logikę w Javie, zdecydowano się zoptymalizować samo zapytanie (query) do bazy danych.
3. **Kompromisy architektoniczne:** Aby przyspieszyć dostarczenie frontendowej części, zrezygnowano ze skomplikowanego systemu zarządzania stanem na rzecz prostszego Context API wbudowanego w Reacta.

11. Podsumowanie

Projekt informatyczny „MyTutor - Szkoła Korepetycji” zakończył się sukcesem technicznym. Osiągnięto wszystkie założone cele podstawowe, dostarczając funkcjonalne środowisko do zarządzania użytkownikami, zajęciami i salami.

Dzięki zastosowaniu nowoczesnego i bezpiecznego stosu technologicznego (Java Spring Boot + React TypeScript), system gwarantuje dużą stabilność oraz łatwość w obsłudze. Zastosowane kompromisy (m.in. uproszczona autoryzacja oraz brak zewnętrznych powiadomień) pozwoliły na terminowe dostarczenie stabilnej wersji MVP.